SpotFakePhoto

Real photo vs. screen-recapture classification - short technical report

Original validation	External holdout	Warm CPU latency	Estimated cost
92.0% (23/25)	75.0% (6/8)	203.86 ms/image	Approx. $0/image

Approach

I treated the task as binary image classification: class 0 is an original photograph and class 1 is a photograph recaptured from a display. The system uses two ImageNet-pretrained MobileNetV3 Small networks. Transfer learning was chosen because the available dataset is too small to train a deep visual model reliably from scratch.

The two networks inspect the image at different scales. The native-detail branch takes small crops without first shrinking a high-resolution image, which helps preserve moire, scanlines, pixel grids and other fine display artifacts. The global-context branch first resizes the image and examines five larger-context crops, making it more sensitive to screen borders, glare, perspective distortion and framing. Their screen probabilities are blended, with 79% weight on native detail and 21% on global context. Training used random resized crops, horizontal flips, small rotations, colour jitter, label smoothing, AdamW, cosine learning-rate decay and early stopping.

Dataset And Evaluation

The adapted training pool contains 71 unique real images and 69 unique screen-recapture images. Sixteen newly collected, lower-resolution examples were included for adaptation, while eight additional new images were kept untouched as an external holdout. No public dataset was added.

On the original fixed validation split, the final ensemble achieved 92.00% accuracy (23/25), with 91.67% precision, 91.67% recall and 91.67% F1. On the new external holdout it achieved 75.00%. I report both numbers because the external score is a more realistic indication of performance on unfamiliar devices and compressed images.

Latency, Deployment And Cost

Average warm inference time was 203.86 ms per image on an Intel Core 7 150U CPU. This excludes Python and PyTorch process startup. The model runs locally, so there is no per-request API charge and the estimated inference cost is approximately $0 per image, apart from electricity and device ownership. A browser demo captures full-resolution webcam frames and sends them only to the local Python server.

Limitations And Improvements

The main limitation is data diversity. Screen recapture changes with display technology, camera sensor, focus distance, angle, lighting and messaging-app compression. The camera demo is therefore less reliable than the internal validation score suggests. The next improvements should be a larger device-separated test set, more hard negatives, balanced examples from several phones and monitors, compression-aware augmentation and probability calibration. For deployment, quantization could reduce latency and model size, followed by monitoring and periodic retraining on real failure cases.